DEEPFAKE DETECTION

Architecture Diagram

System Architecture with Component Explanations

Major Prabhat Pandey | April 2026 | MLOps Project

1. System Architecture Overview

The Deepfake Detection system is a full-stack MLOps platform composed of five distinct architectural layers. Each layer is independent and communicates through well-defined interfaces, ensuring loose coupling and high cohesion.

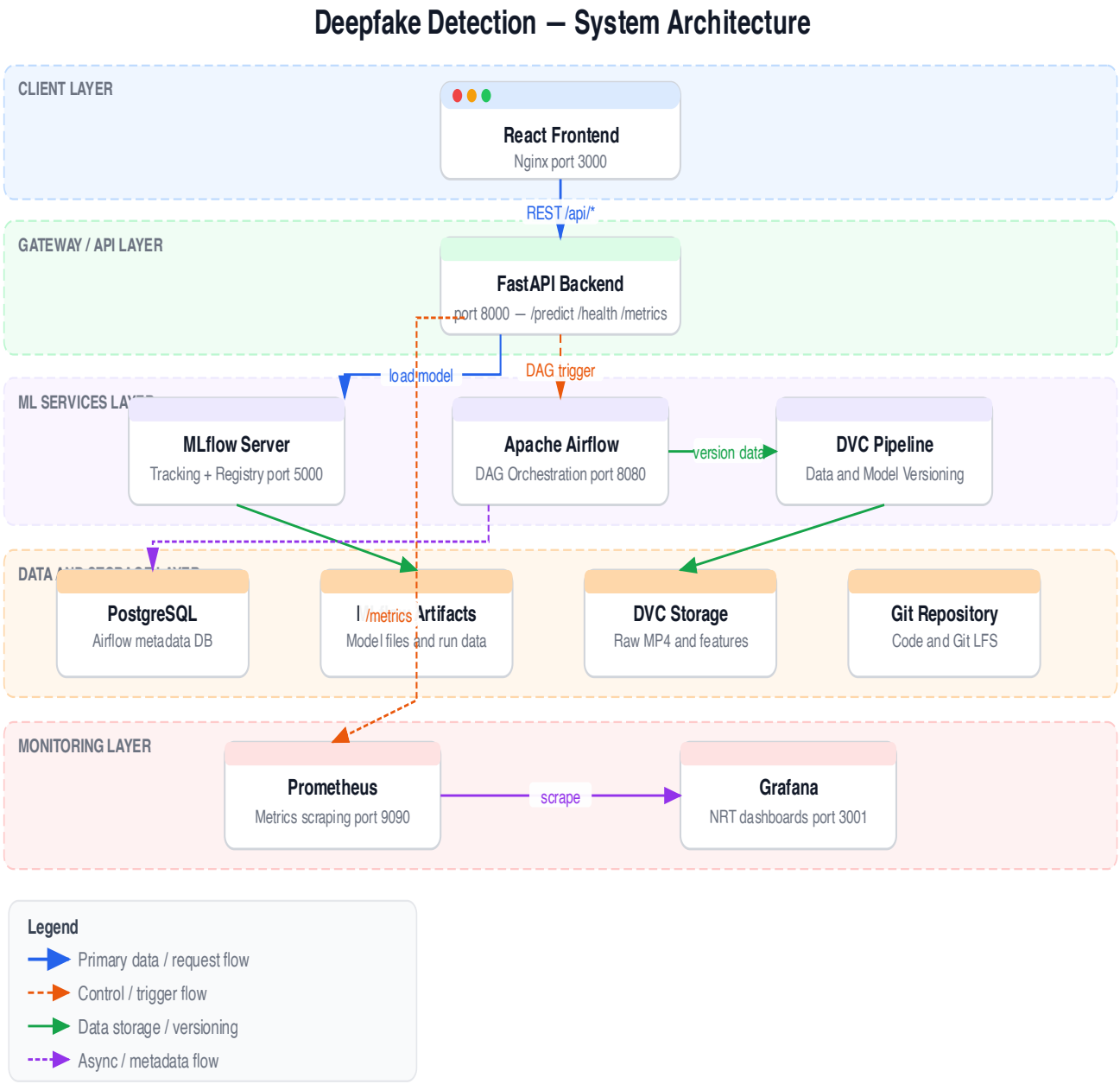

Figure 1: System Architecture — Five-Layer View

1.1 Architecture Layers

The system is organized into five horizontal layers, each with a distinct responsibility:

Layer 1 — Client Layer

The React frontend (served by Nginx on port 3000) is the only user-facing component. It accepts MP4 video uploads via drag-and-drop and presents prediction results including confidence scores and Grad-CAM heatmaps. Nginx proxies all /api/* requests to the backend.

Layer 2 — Gateway / API Layer

FastAPI (port 8000) is the sole entry point for all backend operations. It enforces strict input validation, orchestrates preprocessing, calls the ML model, generates explainability overlays, and exposes Prometheus metrics. The frontend and backend are connected exclusively via REST API — no shared state or direct imports.

Layer 3 — ML Services Layer

Three services power the ML lifecycle:

- MLflow Server (port 5000): Tracks experiments, manages the model registry, and stores training artifacts. The backend loads the production model via mlflow.pyfunc at startup.
- Apache Airflow (port 8080): Orchestrates the data ingestion pipeline (daily) and model retraining pipeline (weekly) using a LocalExecutor — no separate worker container required.
- DVC Pipeline: Manages reproducible data versioning across five stages: extract_frames → detect_faces → compute_features → train → evaluate.

Layer 4 — Data and Storage Layer

Persistent storage is split by concern:

Store	Purpose	Technology
PostgreSQL (port 5432)	Airflow metadata, task state, DAG history	PostgreSQL 15
MLflow Artifacts	Model weights, confusion matrices, ROC curves	Local filesystem / MLflow store
DVC Storage	Raw MP4s, extracted frames, face crops, feature tensors	DVC + Git LFS
Git Repository	Source code, config, dvc.lock, dvc.yaml	Git + Git LFS

Layer 5 — Monitoring Layer

Prometheus (port 9090) scrapes metrics from all four services every 15 seconds. Grafana (port 3001) visualises these in three dashboards: inference performance, model drift, and pipeline health. Alert rules fire when error rate exceeds 5%, p95 latency exceeds 200ms, or drift score exceeds 3.0.

1.2 Loose Coupling Guarantee

The architecture strictly enforces loose coupling at every boundary:

Boundary	Communication Method	Configurable Via
Frontend ↔ Backend	REST API only (/api/* proxy)	VITE_API_URL env var
Backend ↔ ML Model	mlflow.pyfunc.load_model() URI	MODEL_NAME / MODEL_STAGE env vars
Backend ↔ Airflow	HTTP (Airflow REST API)	AIRFLOW_URL env var
Backend ↔ MLflow	HTTP (MLflow Tracking API)	MLFLOW_TRACKING_URI env var
All services ↔ Secrets	.env file (never committed)	.env.example template

1.3 Docker Compose Services

Service	Port	Purpose
frontend	3000	React app + Nginx proxy
backend	8000	FastAPI inference + monitoring
mlflow-server	5000	Experiment tracking + model registry
airflow-webserver	8080	DAG management UI
airflow-scheduler	—	LocalExecutor task runner
postgres	5432	Airflow metadata database
prometheus	9090	Metrics collection
grafana	3001	Dashboards and alerting
node-exporter	9100	Host system metrics